
From struct to class

Ce TD a pour objectif de vous faire découvrir la notion de classe en partant d'une structure de données (struct), qui est la construction du langage C la plus proche du concept de classe.

Structure pour représenter une personne

Pour rappel, une structure (struct) est un élément de langage qui permet de construire un type de données à partir d'autres types nommés **membres** (on parle alors de type composite). L'idée est d'offrir au développeur la possibilité d'étendre le langage pour représenter des éléments qui n'étaient pas initialement prévus.

Ici, nous allons créer une structure Person dont l'objectif est de représenter une personne.

Il est souvent impossible de représenter l'ensemble des informations d'un élément de la vie réelle ; on en représente une approximation suffisamment détaillée pour les besoins du programme.

Copiez le programme suivant et testez-le :

```
#include <iostream>
#include <string>

using namespace std;

struct Person
{
    string firstname;
    string lastname;
    uint age;
    string nationality;
};

int main(int argc, char const *argv[])
{
    Person thomas = {"Thomas", "Le Goff", 27, "Française"};

    cout << thomas.firstname << endl;
    cout << thomas.age << endl;

    return 0;
}
```

Pour compiler et exécuter le programme, vous pouvez utiliser la commande `gcc main.cpp -lstdc++ -o a.out && ./a.out` ou directement `g++ main.cpp && ./a.out`. `g++` est en fait un alias de la commande `gcc` avec un paramétrage adapté au langage C++.

Savoir si la personne est majeure ou non

Une fonctionnalité demandée dans le programme est la possibilité de savoir si une personne est majeure ou non.

En somme, l'adaptation suivante est suffisante :

```
#include <iostream>
#include <string>

using namespace std;

struct Person
{
    string firstname;
    string lastname;
    uint age;
    string nationality;
};

int main(int argc, char const *argv[])
{
    Person thomas = {"Thomas", "Le Goff", 27, "Française"};

    cout << thomas.firstname << endl;
    cout << thomas.age << endl;

    if (thomas.age >= 18) {
        cout << thomas.age << endl;
    }

    return 0;
}
```

Même si cette version est parfaitement fonctionnelle, une problématique de duplication de code apparaît :

```
#include <iostream>
#include <string>
```

```
using namespace std;
```

```
struct Person
```

```
{  
    string firstname;  
    string lastname;  
    uint age;  
    string nationality;  
};
```

```
int main(int argc, char const *argv[])
```

```
{  
    Person thomas = {"Thomas", "Le Goff", 27, "Française"};  
  
    cout << thomas.firstname << endl;  
    cout << thomas.age << endl;  
  
    if (thomas.age >= 18) {  
        cout << thomas.age << endl;  
    }  
  
    Person martin = {"Martin", "Matin", 10, "Française?"};  
  
    cout << martin.firstname << endl;  
    cout << martin.age << endl;  
  
    if (martin.age >= 18) {  
        cout << martin.age << endl;  
    }  
  
    return 0;  
}
```

L'introduction d'une fonction permet de rapidement corriger ce problème :

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
struct Person
```

```
{  
    string firstname;
```

```

    string lastname;
    uint age;
    string nationality;
};

bool isAnAdult(Person person)
{
    return person.age >= 18;
}

int main(int argc, char const *argv[])
{
    Person thomas = {"Thomas", "Le Goff", 27, "Française"};

    cout << thomas.firstname << endl;
    cout << thomas.age << endl;

    if (isAnAdult(thomas)) {
        cout << thomas.age << endl;
    }

    Person martin = {"Martin", "Matin", 10, "Française?"};

    cout << martin.firstname << endl;
    cout << martin.age << endl;

    if (isAnAdult(martin)) {
        cout << martin.age << endl;
    }

    return 0;
}

```

Pour le moment, rien de nouveau : il s'agit des pratiques offertes par la programmation procédurale et structurée (le langage C). Ici, la non-duplication de code repose sur le fait qu'il n'y a qu'un seul et unique endroit à modifier (la fonction `isAnAdult`) pour changer la règle qui définit si une personne est majeure ou non.

Maintenant, compliquons un peu les choses : notre programme est susceptible d'être utilisé avec des personnes de différents pays (en même temps). On aurait alors des personnes qui viennent, possiblement, de pays où l'âge de la majorité n'est plus 18 ans.

La modification la plus simple que nous pouvons apporter est la suivante :

```
#include <iostream>
#include <string>

using namespace std;

struct Person
{
    string firstname;
    string lastname;
    uint age;
    string nationality;
};

bool isAnAdult(Person person)
{
    if (person.nationality == "Française") {
        return person.age >= 18;
    } else {
        return person.age >= 21;
    }
}

int main(int argc, char const *argv[])
{
    Person thomas = {"Thomas", "Le Goff", 27, "Française"};

    cout << thomas.firstname << endl;
    cout << thomas.age << endl;

    if (isAnAdult(thomas)) {
        cout << thomas.age << endl;
    }

    Person martin = {"Martin", "Matin", 10, "Américaine"};

    cout << martin.firstname << endl;
    cout << martin.age << endl;

    if (isAnAdult(martin)) {
        cout << martin.age << endl;
    }

    return 0;
}
```

```
}
```

Elle soulève cependant quelques problématiques : - Comment faire si notre programme doit gérer tous les pays du monde ? - Que se passe-t-il si un développeur se trompe de nationalité ("Français" ou "FR" au lieu de "Française") ?

Pour éviter ces problématiques, il est nécessaire d'améliorer notre modélisation du concept de nationalité et d'âge de la majorité.

Mieux savoir si une personne est majeure

Pour améliorer notre programme, il est possible d'utiliser des concepts **orientés objets**.

La première modification (possible uniquement en C++) est de transformer la fonction `isAdult` en **méthode** :

```
#include <iostream>
#include <string>

using namespace std;

struct Person
{
    string firstname;
    string lastname;
    uint age;
    string nationality;

    bool isAdult()
    {
        if (this->nationality == "Française") {
            return this->age >= 18;
        } else {
            return this->age >= 21;
        }
    }
};

int main(int argc, char const *argv[])
{
    Person thomas = {"Thomas", "Le Goff", 27, "Française"};

    cout << thomas.firstname << endl;
```

```

    cout << thomas.age << endl;

    if (thomas.isAdult()) {
        cout << thomas.age << endl;
    }

    Person martin = {"Martin", "Matin", 10, "Américaine"};

    cout << martin.firstname << endl;
    cout << martin.age << endl;

    if (martin.isAdult()) {
        cout << martin.age << endl;
    }

    return 0;
}

```

Prenez le temps de bien identifier les changements entre cette version et la version précédente. Ici aussi, on constate que peu de choses ont changé. Un élément important a changé : les appels à la fonction/méthode `isAdult` dans la fonction `main`. À partir de cette version, la définition de la majorité réside dans la méthode `isAdult` ; elle sera la seule à changer si l'on souhaite modifier la règle.

Ce principe se nomme **l'encapsulation** : il s'agit d'un principe fondamental de la programmation orientée objet.

Pour aller plus loin, nous allons introduire une nouvelle structure `Country` qui permettra de porter les informations (qui nous intéressent) en lien avec un pays :

```

#include <iostream>
#include <string>

using namespace std;

struct Country
{
    string name;
    uint legalAge;
};

struct Person
{

```

```
string firstname;
string lastname;
uint age;
Country nationality;

bool isAnAdult()
{
    if (this->nationality.name == "France") {
        return this->age >= 18;
    } else {
        return this->age >= 21;
    }
}

};

int main(int argc, char const *argv[])
{
    Country france = {"France", 18};
    Country usa = {"USA", 21};

    Person thomas = {"Thomas", "Le Goff", 27, france};

    cout << thomas.firstname << endl;
    cout << thomas.age << endl;

    if (thomas.isAnAdult()) {
        cout << thomas.age << endl;
    }

    Person martin = {"Martin", "Matin", 10, usa};

    cout << martin.firstname << endl;
    cout << martin.age << endl;

    if (martin.isAnAdult()) {
        cout << martin.age << endl;
    }

    return 0;
}
```

Notez bien les changements effectués dans la fonction `main` : - Un pays n'est plus uniquement matérialisé par une chaîne de caractères, mais par une structure réutilisable par différentes `Person`. - Ce

n'est plus Person qui porte l'information de l'âge légal, mais Country (une adaptation est cependant nécessaire).

En termes orientés objet, on dit que france est une instance/un objet de Country et que Person est **composé** d'un pays.

```
#include <iostream>
#include <string>

using namespace std;

struct Country
{
    string name;
    uint legalAge;
};

struct Person
{
    string firstname;
    string lastname;
    uint age;
    Country nationality;

    bool isAnAdult()
    {
        return this->age >= this->nationality.legalAge;
    }
};

int main(int argc, char const *argv[])
{
    Country france = {"France", 18};
    Country usa = {"USA", 21};

    Person thomas = {"Thomas", "Le Goff", 27, france};

    cout << thomas.firstname << endl;
    cout << thomas.age << endl;

    if (thomas.isAnAdult()) {
        cout << thomas.age << endl;
    }
}
```

```

Person martin = {"Martin", "Matin", 10, usa};

cout << martin.firstname << endl;
cout << martin.age << endl;

if (martin.isAnAdult()) {
    cout << martin.age << endl;
}

return 0;
}

```

Dans cette version, la règle qui permet de savoir si une personne est majeure dans son pays n'est plus "gravée" dans le code, mais représentée par une donnée. Enfin, `isAnAdult` est une méthode qui ne fait que retourner une information : c'est ce que l'on nomme une propriété dérivée des attributs `age` et `nationality`.

Voici un tableau récapitulatif centré uniquement sur les éléments abordés dans vos exemples précédents, avec une formulation concise et adaptée à votre style :

Tableau comparatif : Structure (paradigme procédural) vs Classe (paradigme objet)

Critère	Structure (struct) – Paradigme procédural	Classe – Paradigme objet
Données	Les données sont appelées membres (ex: <code>firstname</code> , <code>age</code> , <code>nationality</code>).	Les données sont appelées attributs (ex: <code>firstname</code> , <code>age</code> , <code>nationality</code>).
Comportement	Les fonctions externes manipulent les membres (ex: <code>bool isAnAdult(Person p)</code>).	Les méthodes sont intégrées à la structure/classe (ex: <code>bool isAnAdult()</code>).
Accès aux données	Accès direct aux membres depuis l'extérieur (ex: <code>thomas.age</code>).	Accès via <code>this-></code> dans les méthodes (ex: <code>this->age</code>).
Encapsulation	Il n'y a pas d'encapsulation : les membres sont accessibles et modifiables depuis n'importe où.	Encapsulation possible : la logique (ex: règle de majorité) est centralisée dans la méthode (<code>isAnAdult</code>).

Structure (struct) – Paradigme procédural		
Critère		Classe – Paradigme objet
Logique métier	La logique est répartie dans des fonctions externes (risque de duplication).	La logique est centralisée dans la méthode de la structure/classe (ex: <code>isAdult</code> dans <code>Person</code>).
Réutilisabilité	Moins flexible : chaque fonction doit être adaptée pour chaque structure.	Plus flexible : la méthode est liée aux données et réutilisable sans duplication.
Évolution	Modifier la règle de majorité nécessite de changer toutes les fonctions externes qui l'utilisent.	Modifier la règle nécessite de changer uniquement la méthode (<code>isAdult</code>).
Composition	Non abordée dans l'exemple de base.	Utilisation de la composition (ex: <code>Person</code> contient un objet <code>Country</code> pour gérer l'âge légal).

- **Passage de fonctions à méthodes** : La logique (`isAdult`) est **intégrée** à la structure, ce qui évite la duplication et centralise la règle.
- **Encapsulation** : La règle métier (ex: âge légal) est **isolée dans la méthode**, facilitant les modifications futures.
- **Composition** : Introduction d'une structure `Country` pour **déléguer la responsabilité** de l'âge légal, rendant le code plus modulaire.